

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

你只需要注意力

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

目前主流的序列转换模型基于复杂的循环神经网络或卷积神经网络，这些网络包含编码器和解码器。性能最佳的模型还通过注意力机制连接编码器和解码器。我们提出了一种新的简单网络架构——Transformer，它完全基于注意力机制，摒弃了循环和卷积。在两项机器翻译任务上的实验表明，我们的模型在质量上更胜一筹，同时更易于并行化，且训练时间显著缩短。我们的模型在 WMT 2014 英译德任务上取得了 28.4 的 BLEU 值，比现有最佳结果（包括集成模型）提高了 2 个 BLEU 以上。在 WMT 2014 英译法任务上，我们的模型在八个 GPU 上训练 3.5 天后，取得了 41.8 的 BLEU 值，创下了单模型最佳成绩的新纪录，而训练成本仅为文献中最佳模型的一小部分。我们通过成功地将 Transformer 应用于英语成分分析任务（无论训练数据量大小），证明了其在其他任务上的良好泛化能力。

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

1 引言

循环神经网络，特别是长短期记忆网络 (LSTM) 和门控循环神经网络 (GRT)，已被公认为序列建模和转换问题（例如语言建模和机器翻译）领域的最先进方法。此后，众多研究不断拓展循环语言模型和编码器-解码器架构的边界。

循环模型通常将计算分解为输入和输出序列的符号位置。通过将位置与计算时间步长对齐，它们生成一个隐藏状态序列，该序列是前一个隐藏状态和位置的输入的函数。这种固有的顺序性限制了训练样本内部的并行化，而当序列长度较长时，并行化变得至关重要，因为内存限制会阻碍跨样本的批处理。最近的研究通过因子分解技巧和条件计算显著提高了计算效率，并在条件计算的情况下也提升了模型性能。然而，顺序计算的根本限制依然存在。

注意力机制已成为各种任务中引人注目的序列建模和转换模型不可或缺的一部分，它允许对输入或输出序列中的依赖关系进行建模，而无需考虑它们之间的距离。然而，除了少数情况外，这种注意力机制通常与循环神经网络结合使用。

本文提出了一种名为 Transformer 的模型架构，它摒弃了循环，而是完全依赖注意力机制来建立输入和输出之间的全局依赖关系。Transformer 模型能够显著提高并行化程度，只需在八块 P100 GPU 上训练十二小时，即可达到翻译质量的新高度。

2 背景

减少顺序计算的目标也是扩展神经 GPU ([16])、ByteNet ([18]) 和 ConvS2S ([9]) 的基础。这些模型都使用卷积神经网络作为基本构建模块，并行计算所有输入和输出位置的隐藏表示。在这些模型中，关联两个任意输入或输出位置的信号所需的运算次数会随着位置间距离的增加而增长，ConvS2S 呈线性增长，ByteNet 呈对数增长。这使得学习远距离位置之间的依赖关系变得更加困难 ([12])。在 Transformer 中，这被简化为恒定次数的操作，但代价是由于对注意力加权位置进行平均而导致有效分辨率降低，我们使用第 3.2 节中描述的多头注意力机制来抵消这种影响。

自注意力（有时也称为序列内注意力）是一种注意力机制，它将单个序列的不同位置关联起来，以计算该序列的表示。自注意力已成功应用于各种任务，包括阅读理解、抽象式摘要、文本蕴含和学习与任务无关的句子表示。

端到端记忆网络基于循环注意力机制而非序列对齐的循环，并且已被证明在简单语言问答和语言建模任务中表现良好。

据我们所知，Transformer 是第一个完全依赖自注意力机制来计算输入和输出表示的转换模型，它无需使用序列对齐的 RNN 或卷积。在接下来的章节中，我们将描述 Transformer，阐述自注意力机制的动机，并讨论它相对于 [17, 18] 和 [9] 等模型的优势。

3 模型架构

大多数具有竞争力的神经序列转换模型都具有编码器-解码器结构 [5, 2, 35]。其中，编码器将符号表示的输入序列 $(x_1, \dots, x_n)$ 映射到连续表示序列 $z = (z_1, \dots, z_n)$ 。给定 z ，解码器随后逐个生成符号的输出序列 $(y_1, \dots, y_m)$ 。在每一步中，模型都是自回归的 [10]，在生成下一个符号时，会将先前生成的符号作为附加输入。

Transformer 遵循此整体架构，编码器和解码器均采用堆叠式自注意力机制和逐点全连接层，分别如图 1 的左半部分和右半部分所示。

3.1 编码器和解码器堆叠

编码器由 $N = 6$ 个相同的层堆叠而成。每个层包含两个子层。第一个子层是多头自注意力机制，第二个子层是一个简单的逐位置全连接前馈网络。我们在每个子层周围使用残差连接 [11]，然后进行层归一化 [1]。也就是说，每个子层的输出是 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 是子层自身实现的函数。为了便于这些残差连接，模型中的所有子层以及嵌入层都产生 $d_{\text{model}} = 512$ 维度的输出。

解码器也由

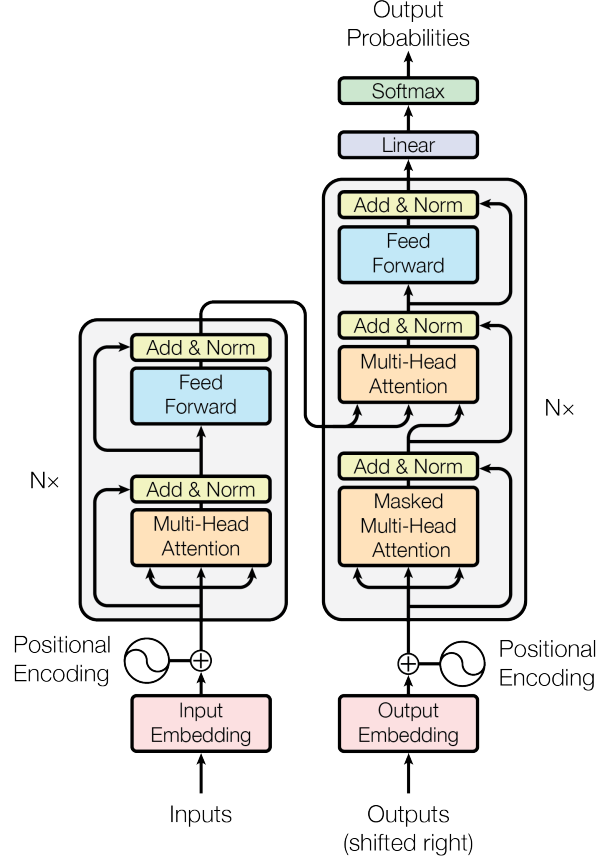


Figure 1: Transformer 模型架构。

解码器： 层堆叠而成，每个编码器层包含相同的 $N = 6$ 个子层。除了每个编码器层中的两个子层之外，解码器还插入了第三个子层，该子层对编码器堆叠的输出执行多头注意力机制。与编码器类似，我们在每个子层周围都使用了残差连接，然后进行层归一化。我们还修改了解码器堆叠中的自注意力子层，以防止位置关注后续位置。这种掩码，结合输出嵌入偏移一个位置的事实，确保了位置 i 的预测只能依赖于位置小于 i 的已知输出。

3.2 注意力

注意力函数可以描述为将查询和一组键值对映射到输出，其中查询、键、值和输出都是向量。输出计算为值的加权和，其中分配给每个值的权重由查询与相应键的兼容性函数计算得出。

3.2.1 缩放点积注意力

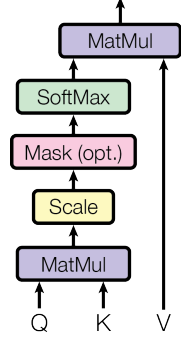
我们将我们提出的注意力称为“缩放点积注意力”（图 2）。输入由维度为 d_k 的查询和键，以及维度为 d_v 的值组成。我们计算查询与所有键的点积，并将每个点积除以 $\sqrt{d_k}$ ，然后应用 softmax 函数来获得值的权重。

在实践中，我们同时对一组查询计算注意力函数，并将这些查询打包成矩阵 Q 。键和值也打包成矩阵 K 和 V 。我们计算输出矩阵如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

两种最常用的注意力函数是加性注意力 [2] 和点积（乘性）注意力。点积注意力与我们的算法相同，只是缩放因子为 $\frac{1}{\sqrt{d_k}}$ 。加性注意力使用具有单个隐藏层的前馈网络来计算兼容性

Scaled Dot-Product Attention



Multi-Head Attention

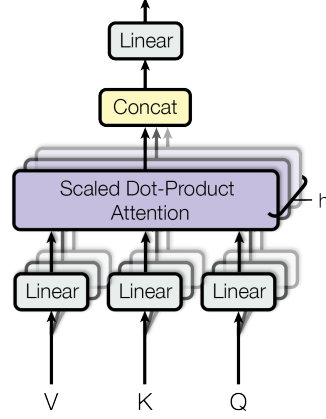


Figure 2: (左) 缩放点积注意力机制。(右) 多头注意力机制由多个并行运行的注意力层组成。

函数。虽然两者在理论上的复杂度相似，但点积注意力机制在实践中速度更快、空间效率更高，因为它可以使用高度优化的矩阵乘法代码来实现。

对于较小的 d_k 值，两种机制的性能相似；但对于较大的 d_k [3] 值，加性注意力机制的性能优于未进行缩放的点积注意力机制。我们推测，对于较大的 d_k 值，点积的绝对值会增大，从而将 softmax 函数推入梯度极小的区域⁴。为了抵消这种影响，我们将点积乘以 $\frac{1}{\sqrt{d_k}}$ 进行缩放。

3.2.2 多头注意力机制

我们发现，与其对 d_{model} 维的键、值和查询执行单一的注意力函数，不如将查询、键和值分别使用不同的、已学习的线性投影进行 h 次线性投影，分别到 d_k 、 d_k 和 d_v 维，这样效果更好。然后，我们对每个投影后的查询、键和值并行执行注意力函数，得到 d_v 维的输出值。将这些输出值连接起来并再次投影，得到最终值，如图 2 所示。

多头注意力机制允许模型联合关注来自不同表示子空间、位于不同位置的信息。而使用单个注意力头时，平均操作会抑制这种联合关注。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

其中投影是参数矩阵 $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ 和 $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 。

在这项工作中，我们采用了 $h = 8$ 个并行注意力层，或称注意力头。每个注意力头都使用 $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个注意力头的维度都降低了，因此总计算成本与全维度的单头注意力类似。

3.2.3 注意力在我们模型中的应用

Transformer 以三种不同的方式使用多头注意力：

⁴为了说明点积为何会增大，假设 q 和 k 的分量是均值为 0、方差为 1 的独立随机变量。那么它们的点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为 0，方差为 d_k 。

- 在“编码器-解码器注意力”层中，查询来自前一个解码器层，记忆键和值来自编码器的输出。这使得解码器中的每个位置都可以关注输入序列中的所有位置。这模拟了序列到序列模型（例如 [38, 2, 9]）中典型的编码器-解码器注意力机制。
- 编码器包含自注意力层。在自注意力层中，所有键、值和查询都来自同一位置，在本例中，即编码器中前一层的输出。编码器中的每个位置都可以关注编码器前一层中的所有位置。
- 类似地，解码器中的自注意力层允许解码器中的每个位置关注解码器中直至该位置的所有位置。我们需要阻止解码器中信息的左向流动，以保持自回归特性。我们在缩放点积注意力机制中实现了这一点，方法是屏蔽掉（设置为 $-\infty$ ）softmax 输入中所有对应于非法连接的值。参见图 2。

3.3 位置前馈网络

除了注意力子层之外，我们的编码器和解码器中的每一层都包含一个全连接前馈网络，该网络分别且相同地应用于每个位置。它由两个线性变换组成，中间使用 ReLU 激活函数。

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

虽然不同位置的线性变换相同，但它们在每一层使用的参数都不同。另一种描述方式是将其视为两个卷积核大小为 1 的卷积。输入和输出的维度为 $d_{\text{model}} = 512$ ，而内层的维度为 $d_{\text{ff}} = 2048$ 。

3.4 嵌入和 Softmax

与其他序列转换模型类似，我们使用学习到的嵌入将输入标记和输出标记转换为 d_{model} 维度的向量。我们还使用常用的学习到的线性变换和 softmax 函数将解码器输出转换为预测的下一个标记的概率。在我们的模型中，两个嵌入层和预 softmax 线性变换共享同一个权重矩阵，类似于 [30]。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\text{model}}}$ 。

由于我们的模型不包含递归和卷积，为了使模型能够利用序列的顺序，我们必须注入一些关于序列中标记的相对或绝对位置的信息。为此，我们在编码器和解码器堆栈的底部为输入嵌入添加“位置编码”。位置编码与嵌入具有相同的维度 d_{model} ，因此两者可以相加。位置编码有很多选择，可以是学习得到的，也可以是固定的 [9]。

在这项工作中，我们使用不同频率的正弦和余弦函数：

$$\begin{aligned} PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{\text{model}}}) \\ PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{\text{model}}}) \end{aligned}$$

其中 pos 是位置， i 是维度。也就是说，位置编码的每个维度都对应一个正弦波。波长从 2π 到 $10000 \cdot 2\pi$ 构成等比数列。我们选择此函数是因为我们假设它能让模型更容易地学习通过相对位置进行注意力控制，因为对于任何固定的偏移量 k ， PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。

我们也尝试使用学习到的位置嵌入 [9]，发现这两个版本产生了几乎相同的结果（参见表 3 第 (E) 行）。我们选择正弦版本是因为它可能允许模型外推到比训练期间遇到的序列长度更长的序列长度。

4 为什么需要自注意力机制

在本节中，我们将自注意力层与循环层和卷积层进行比较，后者常用于将一个变长符号表示序列 $(x_1, \dots, x_n)$ 映射到另一个等长序列 $(z_1, \dots, z_n)$ ，其中 $x_i, z_i \in \mathbb{R}^d$ 为序列，例如典型序列转换编码器或解码器中的隐藏层。为了说明我们使用自注意力层的动机，我们考虑了三个关键因素。

第一个因素是每层的总计算复杂度。第二个因素是可并行化的计算量，以所需的最小顺序操作数来衡量。

Table 1: 不同层类型的最大路径长度、每层复杂度和最小顺序操作次数。 n 为序列长度, d 为表示维度, k 为卷积核大小, r 为受限自注意力机制中邻域的大小。

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

第三个因素是网络中长程依赖关系之间的路径长度。学习长程依赖关系是许多序列转换任务的关键挑战。影响学习此类依赖关系能力的一个关键因素是前向和后向信号在网络中必须经过的路径长度。输入和输出序列中任意位置组合之间的路径越短, 就越容易学习长程依赖关系 [12]。因此, 我们也比较了由不同层类型构成的网络中任意两个输入和输出位置之间的最大路径长度。

如表 1 所示, 自注意力层使用固定数量的顺序执行操作连接所有位置, 而循环层则需要 $O(n)$ 次顺序操作。就计算复杂度而言, 当序列长度 n 小于表示维度 d 时, 自注意力层比循环层更快。这种情况在机器翻译中最先进的模型所使用的句子表示中最为常见, 例如词段表示 [38] 和字节对表示 [31]。为了提高处理超长序列任务的计算性能, 可以将自注意力限制在输入序列中以相应输出位置为中心、大小为 r 的邻域内。这将使最大路径长度增加到 $O(n/r)$ 。我们计划在未来的工作中进一步研究这种方法。

单个卷积层, 其卷积核宽度为 $k < n$, 并不能连接所有输入和输出位置对。这样做需要堆叠 $O(n/k)$ 个卷积层 (对于连续卷积核), 或者堆叠 $O(\log_k(n))$ 个卷积层 (对于空洞卷积 [18]), 从而增加网络中任意两点之间最长路径的长度。卷积层的复杂度通常比循环层高 k 倍。然而, 可分离卷积 [6] 可以显著降低复杂度至 $O(k \cdot n \cdot d + n \cdot d^2)$ 。即使使用 $k = n$, 可分离卷积的复杂度也等于自注意力层和逐点前馈层的组合, 这正是我们在模型中采用的方法。

此外, 自注意力机制还能产生更易于解释的模型。我们在附录中考察了模型的注意力分布, 并展示和讨论了相关示例。不仅各个注意力头能够清晰地学习执行不同的任务, 许多注意力头还表现出与句子句法和语义结构相关的行为。

本节描述了我们模型的训练方案。

我们使用标准的 WMT 2014 英德数据集进行训练, 该数据集包含约 450 万个句子对。句子采用字节对编码 ([3]) 进行编码, 其源-目标词汇表包含约 37000 个词元。对于英法翻译, 我们使用了规模更大的 WMT 2014 英法数据集, 该数据集包含 3600 万个句子, 并将词元拆分为 32000 个词段的词汇表 ([38])。句子对按大致序列长度进行批量处理。每个训练批次包含一组句子对, 其中约有 25000 个源词元和 25000 个目标词元。

4.1 硬件和进度安排

我们在一台配备 8 个 NVIDIA P100 GPU 的机器上训练模型。对于使用本文所述超参数的基础模型, 每个训练步骤耗时约 0.4 秒。基础模型总共训练了 100,000 个步骤, 耗时 12 小时。对于我们的大型模型 (详见表 3 的最后一行), 每个步骤耗时 1.0 秒。大型模型训练了 300,000 个步骤 (3.5 天)。

4.2 优化器

我们使用 Adam 优化器 [20] 以及 $\beta_1 = 0.9$ 、 $\beta_2 = 0.98$ 和 $\epsilon = 10^{-9}$ 。我们在训练过程中根据以下公式调整学习率:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

这相当于在前 $warmup_steps$ 个训练步骤中线性增加学习率, 之后按步骤数的平方根倒数比例递减。我们使用了 $warmup_steps = 4000$ 。

Table 2: Transformer 模型在英德和英法 newstest2014 测试中，以远低于现有最先进模型的训练成本，取得了更高的 BLEU 分数。

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

4.3 正则化

我们在训练过程中采用了三种正则化方法：

残差丢弃 我们在每个子层的输出与子层输入相加并归一化之前，对其应用 dropout [33]。此外，我们还对编码器和解码器堆栈中的嵌入和位置编码之和和应用 dropout。对于基础模型，我们使用 $P_{drop} = 0.1$ 的丢弃率。

标签平滑 在训练过程中，我们采用了 $\epsilon_{ls} = 0.1$ [36] 的标签平滑。这会增加困惑度，因为模型会学习到更多的不确定性，但会提高准确率和 BLEU 分数。

5 结果

5.1 机器翻译

在 WMT 2014 英德翻译任务中，大型 Transformer 模型（表 2 中的 Transformer (big)）的性能优于之前报道的最佳模型（包括集成模型），BLEU 值提升超过 2.0，创造了新的最快 BLEU 值 28.4。该模型的配置列于表 3 的最后一行。训练耗时 3.5 天，使用 8 个 P100 GPU。即使是我们的基础模型，其性能也超越了所有已发表的模型和集成模型，而训练成本却远低于任何竞争模型。

在 WMT 2014 英译法任务上，我们的大型模型取得了 41.0 的 BLEU 分数，优于所有已发表的单模型，且训练成本低于之前最先进模型的 1/4。用于英译法的 Transformer（大型）模型使用了 $P_{drop} = 0.1$ 的 dropout 率，而非 0.3。

对于基础模型，我们使用了通过对最近 5 个检查点（每隔 10 分钟写入一次）取平均值得到的单个模型。对于大型模型，我们取了最近 20 个检查点的平均值。我们使用束搜索算法，束大小为 4，长度惩罚为 $\alpha = 0.6$ [38]。这些超参数是在开发集上进行实验后选择的。我们在推理期间将最大输出长度设置为输入长度 + 50，但尽可能提前终止 [38]。

表 2 总结了我们的结果，并将我们的翻译质量和训练成本与文献中的其他模型架构进行了比较。我们通过将训练时间、使用的 GPU 数量以及每个 GPU 的持续单精度浮点运算能力估计值⁵相乘来估算训练模型所需的浮点运算次数。

5.2 模型变体

为了评估 Transformer 模型中不同组件的重要性，我们以不同的方式调整了基础模型，并在开发集 newstest2013 上测量了英德翻译性能的变化。我们使用了上一节中描述的束搜索，但没有使用检查点平均。这些结果呈现在表 3 中。

⁵我们分别对 K80、K40、M40 和 P100 使用了 2.8、3.7、6.0 和 9.5 TFLOPS 的值。

Table 3: Transformer 架构的变体。未列出的值与基础模型相同。所有指标均基于英德翻译开发数据集 newstest2013。列出的困惑度是按词段计算的，基于我们的字节对编码，不应与按词计算的困惑度进行比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$		
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65		
(A)					1	512	512				5.29	24.9		
					4	128	128				5.00	25.5		
					16	32	32				4.91	25.8		
					32	16	16				5.01	25.4		
(B)					16					5.16	25.1	58		
					32					5.01	25.4	60		
(C)	2									6.11	23.7	36		
	4									5.19	25.3	50		
	8									4.88	25.5	80		
		256					32	32				5.75	24.5	28
		1024					128	128				4.66	26.0	168
			1024									5.12	25.4	53
			4096									4.75	26.2	90
(D)							0.0				5.77	24.6		
							0.2				4.95	25.5		
								0.0				4.67	25.3	
								0.2				5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7			
big	6	1024	4096	16					0.3	300K	4.33	26.4	213	

Table 4: Transformer 模型能够很好地泛化到英语成分分析任务（结果见《华尔街日报》第 23 节）。

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

在表 3 的 (A) 行中，我们改变了注意力头的数量以及注意力键和值的维度，同时保持计算量不变，如第 3.2.2 节所述。虽然单头注意力比最佳设置差 0.9 个 BLEU 值，但注意力头过多也会导致翻译质量下降。

在表 3 的第 (B) 行中，我们观察到减小注意力键的大小 d_k 会降低模型质量。这表明确定兼容性并非易事，并且使用比点积更复杂的兼容性函数可能是有益的。我们进一步在第 (C) 行和 (D) 行中观察到，正如预期的那样，更大的模型更好，并且 dropout 在避免过拟合方面非常有效。在第 (E) 行中，我们将正弦位置编码替换为学习到的位置嵌入 [9]，并观察到与基础模型几乎相同的结果。

5.3 英语成分分析

为了评估 Transformer 模型是否能够泛化到其他任务，我们对英语成分分析任务进行了实验。这项任务面临着特殊的挑战：输出受到严格的结构约束，并且比输入长得多。此外，RNN 序列到序列模型在小数据集情况下尚未取得最先进的结果 ([37])。

我们使用宾夕法尼亚树库 (Penn Treebank) 的《华尔街日报》(WSJ) 部分 (约 4 万个训练句子, [25]) 训练了一个 4 层 Transformer 模型 ($d_{model} = 1024$)。我们还使用更大的高置信度语料库和 BerkeleyParser 语料库 (约 1700 万个句子, [37]) 在半监督设置下对其进行了训练。对于仅包含 WSJ 的设置，我们使用了 1.6 万个词元的词汇表；对于半监督设置，我们使用了 3.2 万个词元的词汇表。

我们仅进行了少量实验，以选择 dropout、注意力机制和残差 (第 22 节开发集 4.3)、学习率和束大小，所有其他参数均与英德基础翻译模型保持一致。在推理过程中，我们将最大输出长度增加到输入长度 + 300。对于仅针对 WSJ 的数据集和半监督设置，我们使用了 21 和 $\alpha = 0.3$ 的束大小。

表 4 中的结果表明，尽管缺乏针对特定任务的调优，我们的模型表现却出乎意料地好，其结果优于所有先前报道的模型，仅次于循环神经网络语法 [8]。

与 RNN 序列到序列模型 [37] 相比，即使仅在包含 4 万个句子的 WSJ 训练集上进行训练，Transformer 的性能也优于 BerkeleyParser [29]。

6 结论

在这项工作中，我们提出了 Transformer，这是第一个完全基于注意力机制的序列转换模型，它用多头自注意力机制取代了编码器-解码器架构中最常用的循环层。

对于翻译任务，Transformer 的训练速度明显快于基于循环层或卷积层的架构。在 WMT 2014 英译德和 WMT 2014 英译法翻译任务上，我们都取得了新的最佳结果。在英译德任务中，我们的最佳模型甚至优于所有先前报道的集成模型。

我们对基于注意力机制的模型的未来充满期待，并计划将其应用于其他任务。我们计划将 Transformer 模型扩展到除文本之外的其他输入输出模态问题，并研究局部受限的注意力机制，以高效处理图像、音频和视频等大型输入输出。减少生成过程的序列性也是我们的研究目标之一。

我们用于训练和评估模型的代码可在 <https://github.com/tensorflow/tensor2tensor> 获取。

致谢 我们衷心感谢 Nal Kalchbrenner 和 Stephan Gouws 的宝贵意见、更正和启发。

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.
- [5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.

- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.
- [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.
- [17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.
- [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Łukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.
- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.
- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.

- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.
- [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.
- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

7

注意力可视化

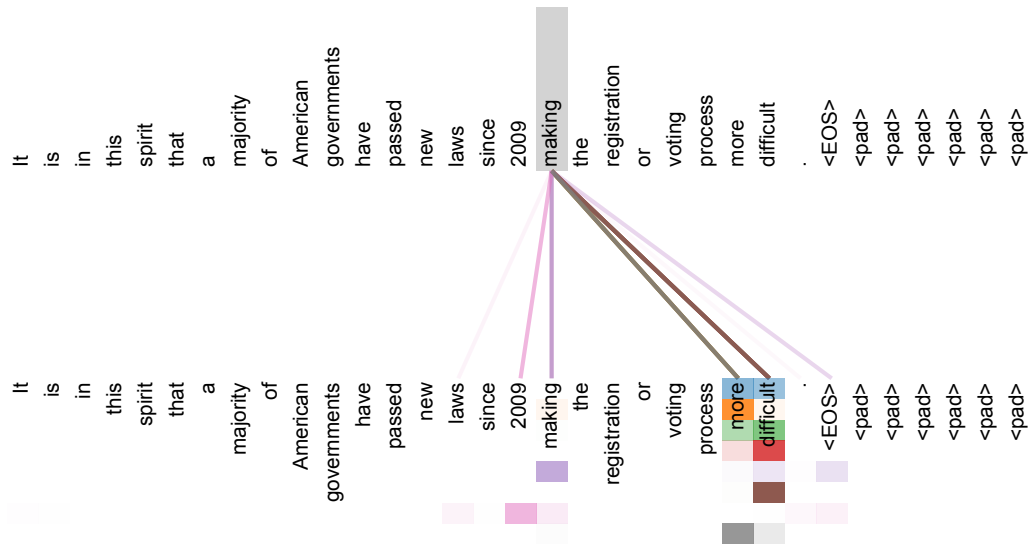


Figure 3: 这是编码器自注意力机制中第 5 层（共 6 层）长距离依赖关系的注意力机制示例。许多注意力头关注单词 “making” 的远距离依赖关系，从而完成短语 “making...more difficult”。此处仅显示单词 “making” 的注意力。不同颜色代表不同的注意力头。建议使用彩色视图查看。

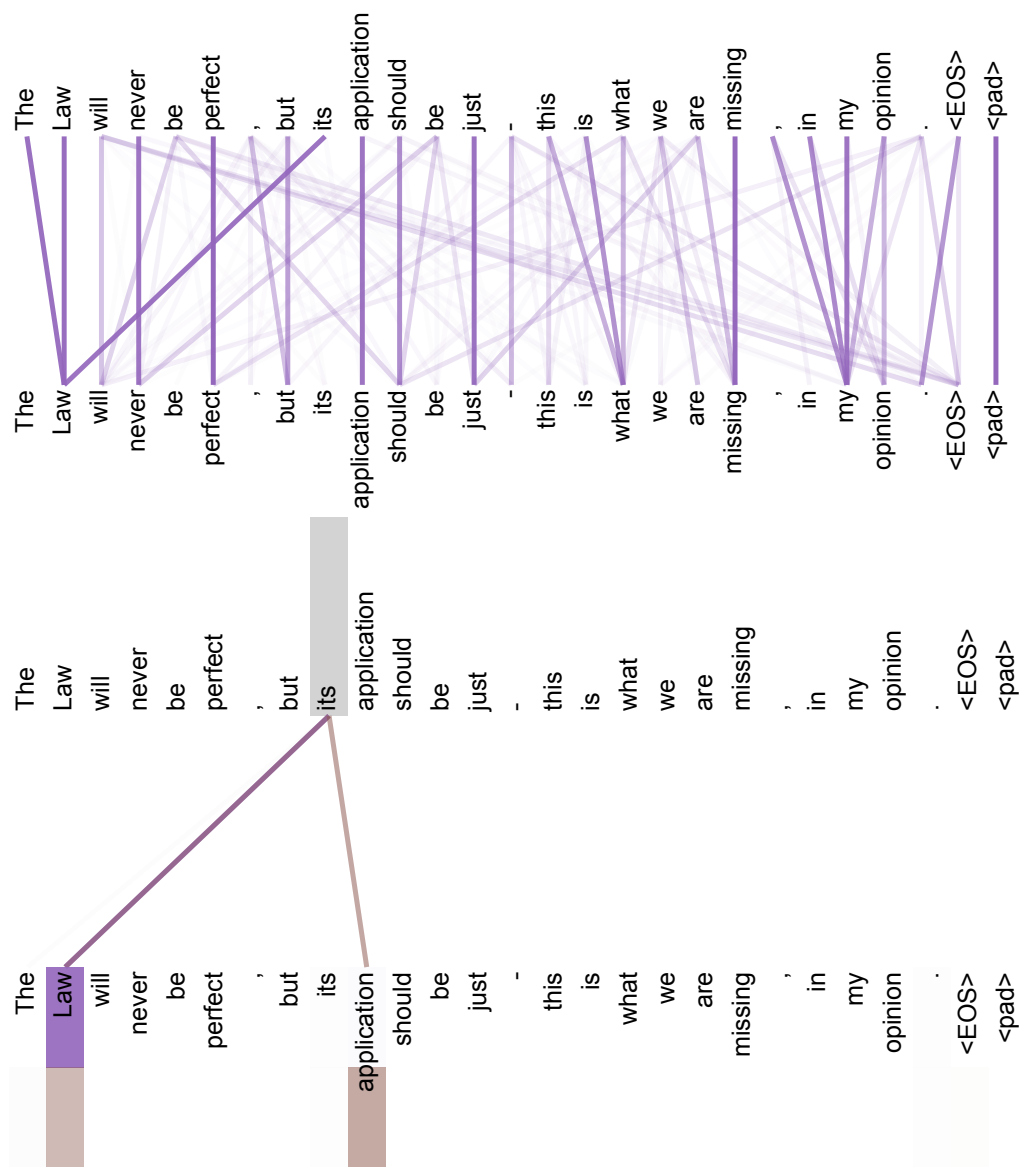


Figure 4: 第5层（共6层）中还有两个注意力头，它们显然参与了指代消解。上图：注意力头5的完整注意力。下图：注意力头5和6仅针对单词“its”的单独注意力。请注意，这些注意力对该词非常敏感。

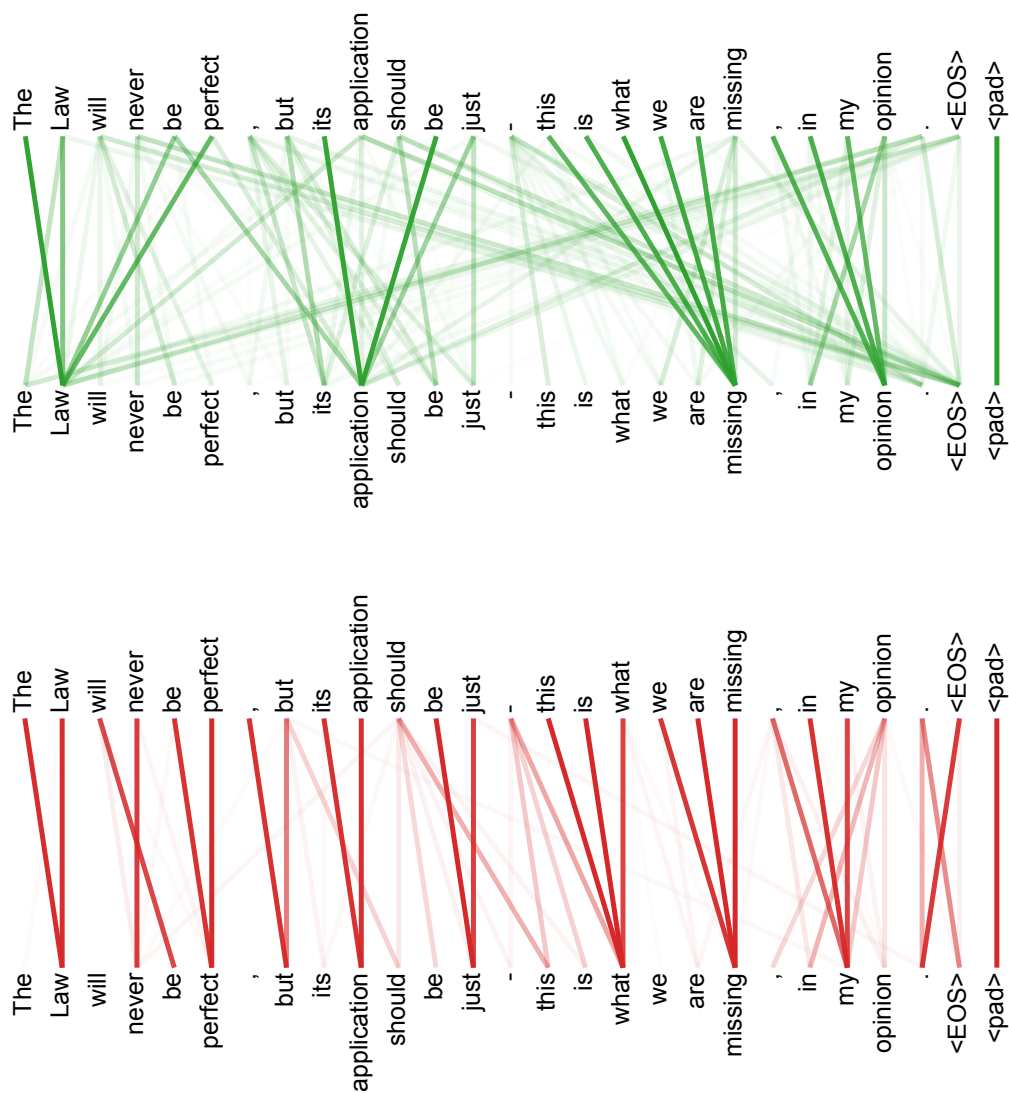


Figure 5: 许多注意力头表现出的行为似乎与句子结构相关。以上我们给出了两个这样的例子，分别来自编码器自注意力机制第 5 层（共 6 层）中的两个不同的注意力头。这些注意力头显然学习执行不同的任务。